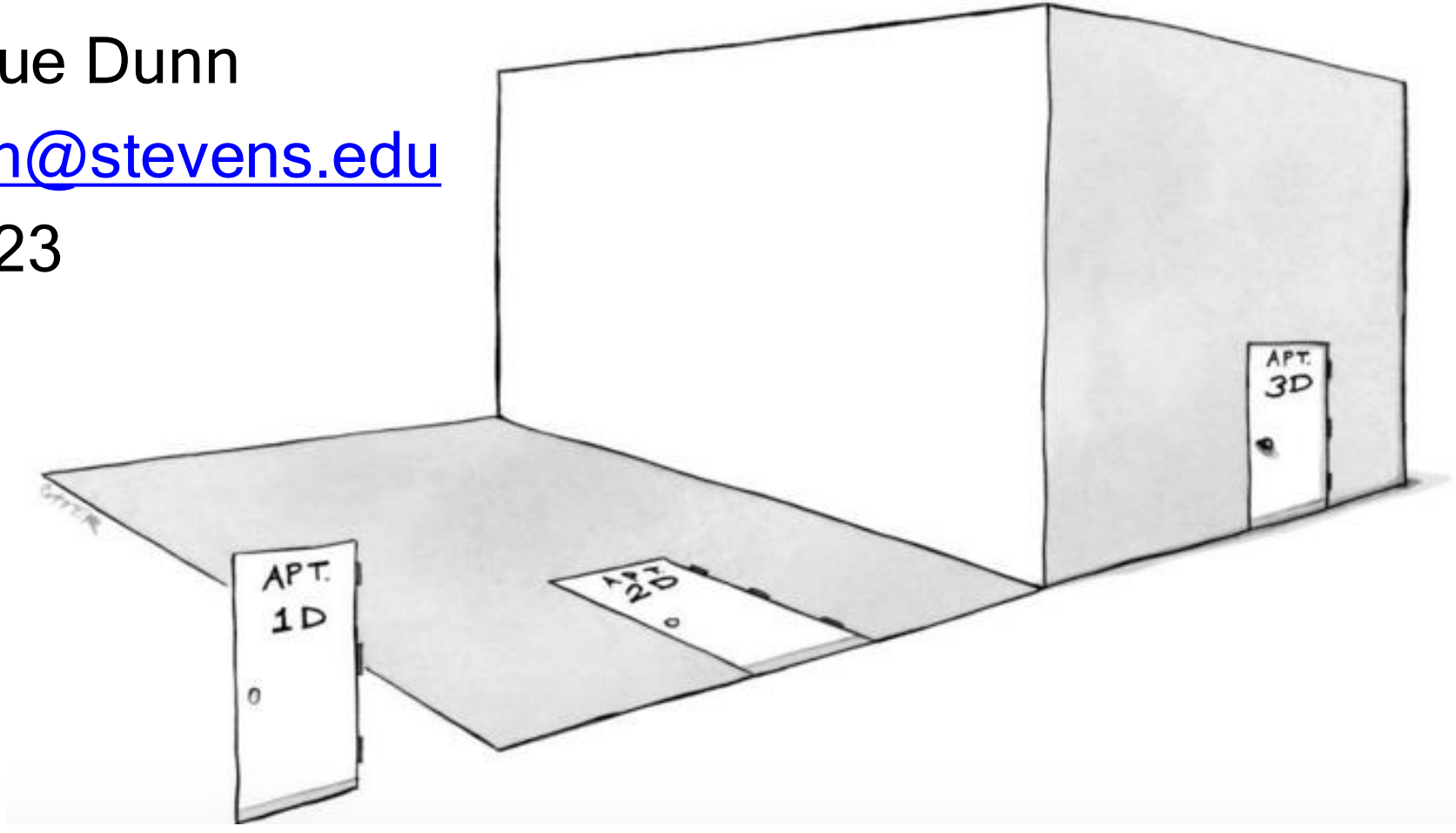
CS 532: 3D Computer Vision
Lecture 6

Enrique Dunn

edunn@stevens.edu

GS 323



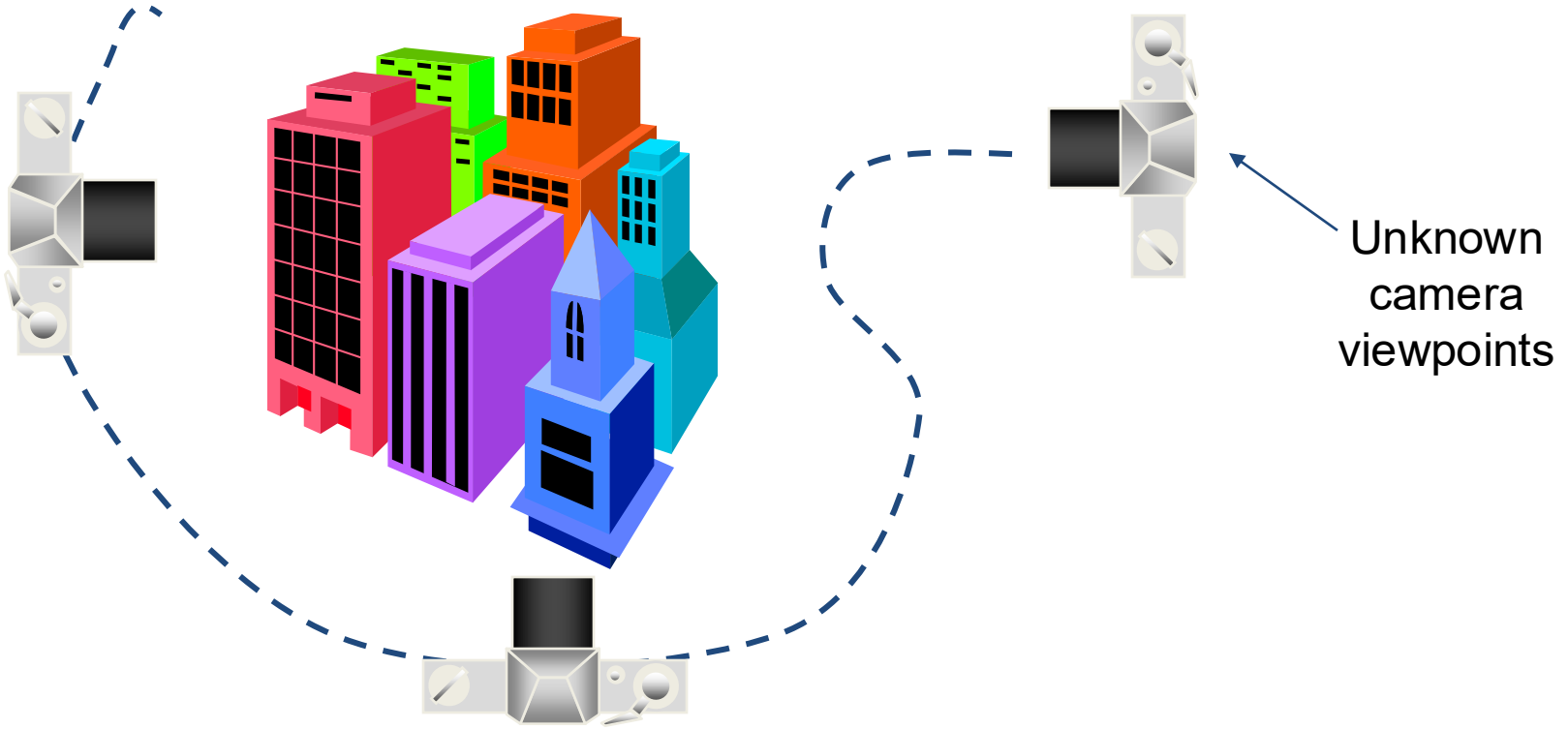
Structure From Motion

Lecture Outline

- Structure from Motion

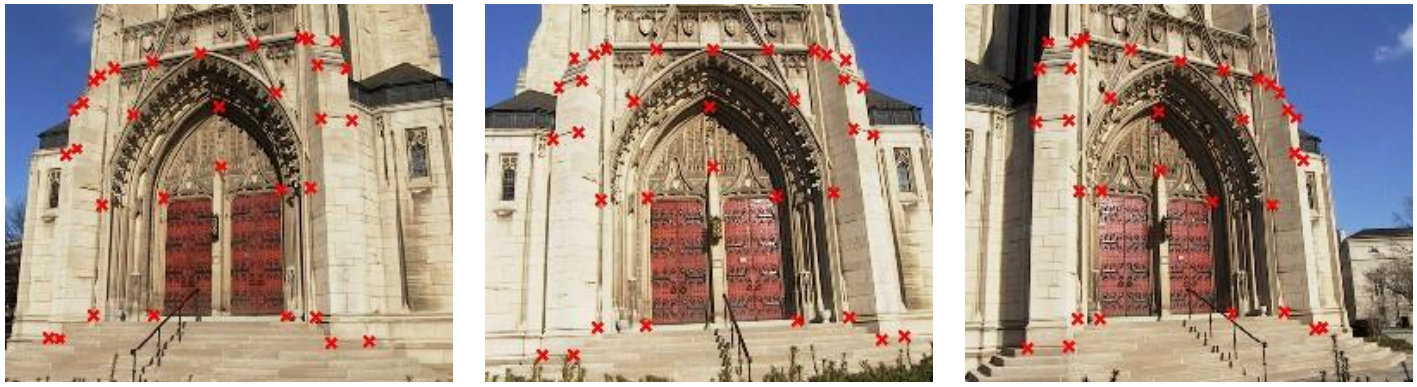
- Sources:
  - Slides by R. Szeliski, S. Seitz, N. Snavely. S. Lazebnik, M. Hebert, S. Choudhary
  - Visual Odometry by D. Nister, O. Naroditsky, J. Bergen (2006)
  - Parallel Tracking and Mapping by G. Klein and D. Murray (2007)
  - Visual SLAM: Why filter? by H. Strasdat, J.M.M. Montiel, A.J. Davison (2012)

Structure from Motion



- Reconstruct
  - Scene geometry
  - Camera motion

Input: Feature Tracks



- Detect good features
  - corners, line segments
- Find correspondences between frames
  - Lucas & Kanade-style motion estimation
  - window-based correlation

Structure from Motion

- Given many points in *correspondence* across several images, $\{(u_{ij}, v_{ij})\}$, simultaneously compute the 3D location $\mathbf{x}_i$ and camera (or *motion*) parameters ($\mathbf{K}$, $\mathbf{R}_j$, $\mathbf{t}_j$)

$$\begin{aligned} \hat{u}_{ij} &= f(\mathrm{K}, \mathrm{R}_j, \mathrm{t}_j, \mathrm{x}_i) \\ \hat{v}_{ij} &= g(\mathrm{K}, \mathrm{R}_j, \mathrm{t}_j, \mathrm{x}_i) \end{aligned}$$

- Two main variants: calibrated, and uncalibrated (sometimes associated with Euclidean and projective reconstructions)

Number of Constraints

$$\hat{u}_{ij} = f(\mathbf{K}, \mathbf{R}_j, \mathbf{t}_j, \mathbf{x}_i)$$
$$\hat{v}_{ij} = g(\mathbf{K}, \mathbf{R}_j, \mathbf{t}_j, \mathbf{x}_i)$$

- How many points do we need to match?
- 2 frames:

  (**R**,**t**): 5 dof + 3n point locations ≤

  4n point measurements ⇒ n ≥ 5

- k frames:

  6(k-1)-1 + 3n ≤ 2kn
- always want to use many more

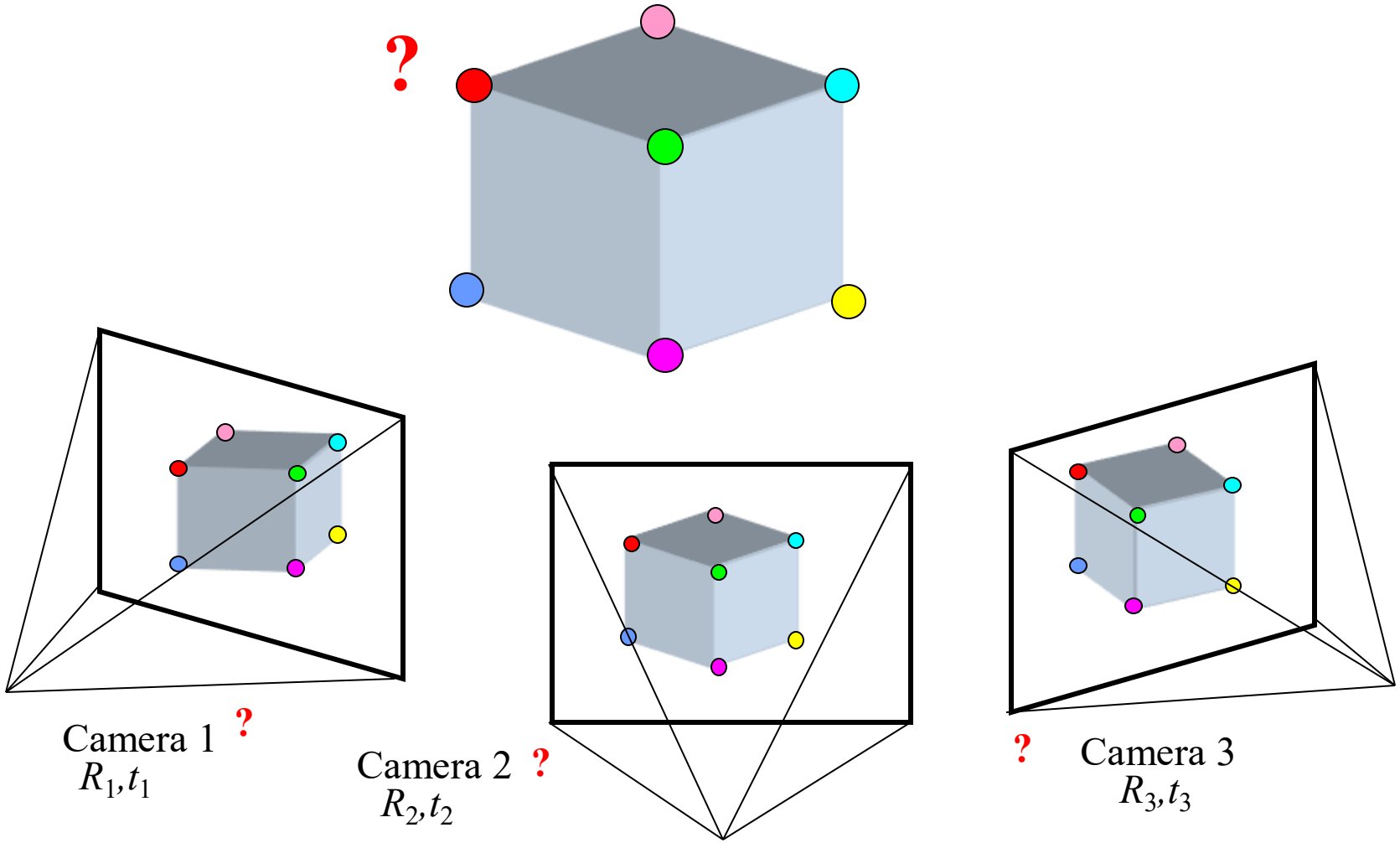
=> why 5 dof for 2 cameras and 6(k-1)-1 for k cameras?

Bundle Adjustment

- What makes this non-linear minimization hard?
  - many parameters: potentially slow
  - poorer conditioning (high correlation)
  - potentially lots of outliers
  - gauge (coordinate) freedom

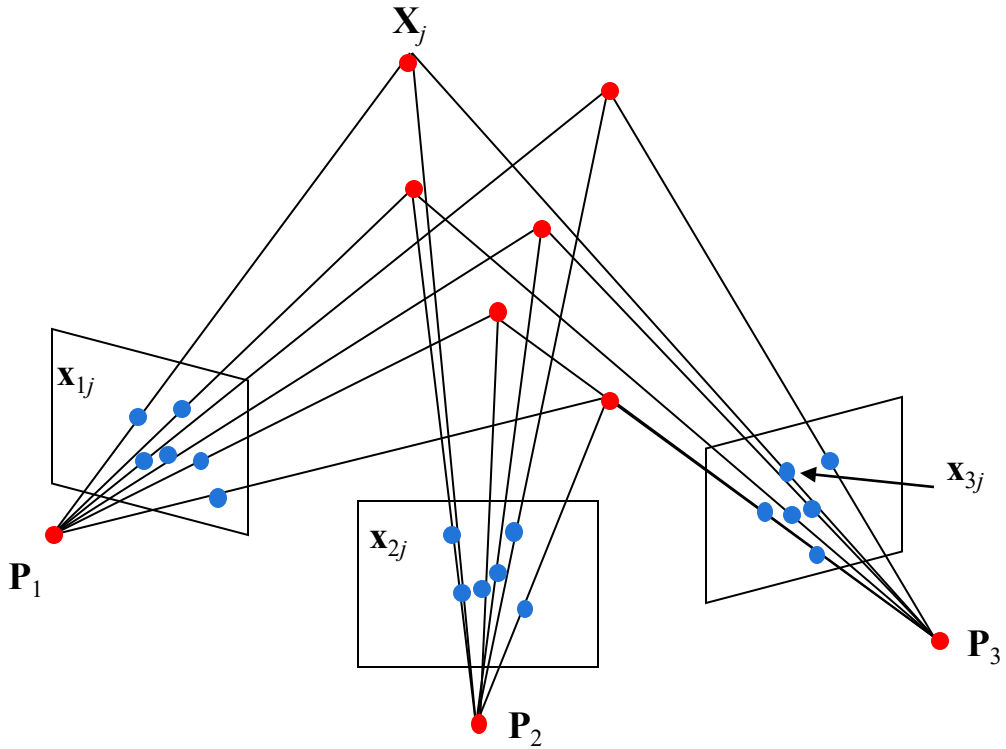
Structure from Motion

- Given a set of corresponding points in two or more images, compute the camera parameters and the 3D point coordinates



Structure from Motion

- Given: *m* images of *n* fixed 3D points
  - $\mathbf{x}_{ij} = \mathbf{P}_i \mathbf{X}_j, \qquad i = 1, \ldots, m, \quad j = 1, \ldots, n$
- Problem: estimate *m* projection matrices $\mathbf{P}_i$ and *n* 3D points $\mathbf{X}_j$ from the *mn* correspondences $\mathbf{x}_{ij}$



Structure from Motion Ambiguity

- If we scale the entire scene by some factor k and, at the same time, scale the camera matrices by the factor of $1/k$, the projections of the scene points in the image remain exactly the same:

$$\mathbf{x} = \mathbf{P}\mathbf{X} = \left(\frac{1}{k}\mathbf{P}\right)(k\mathbf{X})$$

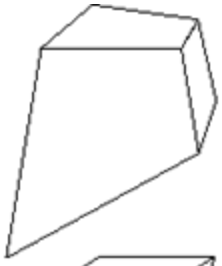
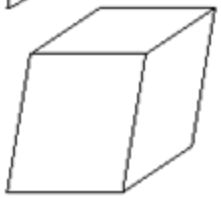
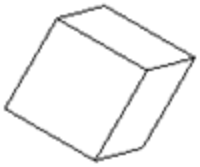
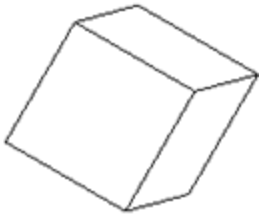
It is impossible to recover the absolute scale of the scene!

Structure from Motion Ambiguity

- More generally: if we transform the scene using a transformation **Q** and apply the inverse transformation to the camera matrices, then the images do not change

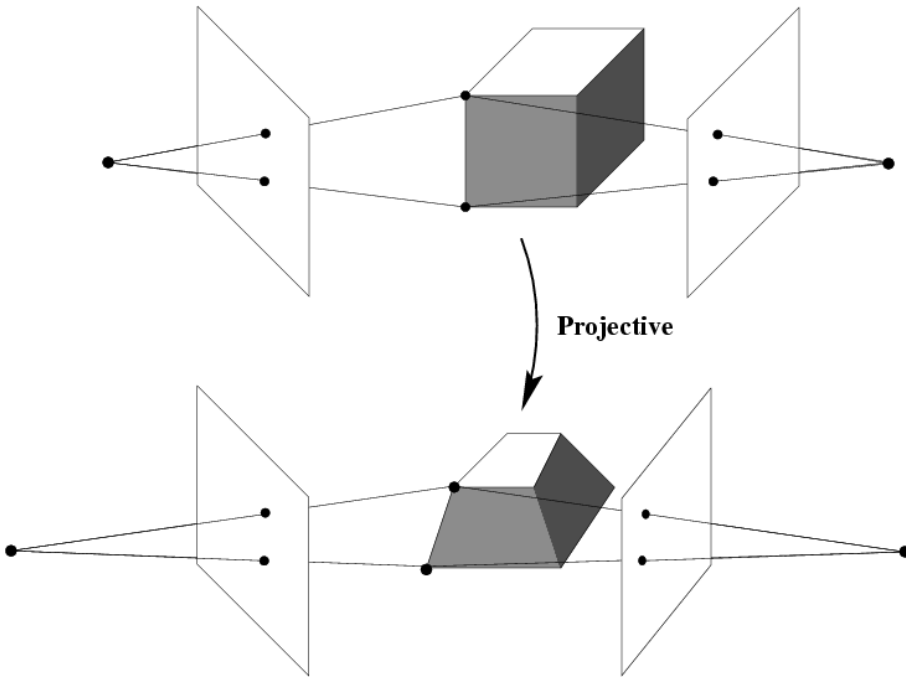
$$\mathbf{x} = \mathbf{PX} = \left(\mathbf{PQ}^{-1}\right)\left(\mathbf{QX}\right)$$

Types of Ambiguity

Projective 15dof	$\begin{bmatrix} A & t \\ v^T & v \end{bmatrix}$		Preserves intersection and tangency
Affine 12dof	$\begin{bmatrix} A & t \\ 0^T & 1 \end{bmatrix}$		Preserves parallellism, volume ratios
Similarity 7dof	$\begin{bmatrix} sR & t \\ 0^T & 1 \end{bmatrix}$		Preserves angles, ratios of length
Euclidean 6dof	$\begin{bmatrix} R & t \\ 0^T & 1 \end{bmatrix}$		Preserves angles, lengths

- With no constraints on the camera calibration matrix or on the scene, we get a *projective* reconstruction
- Need additional information to *upgrade* the reconstruction to affine, similarity, or Euclidean

Projective Ambiguity



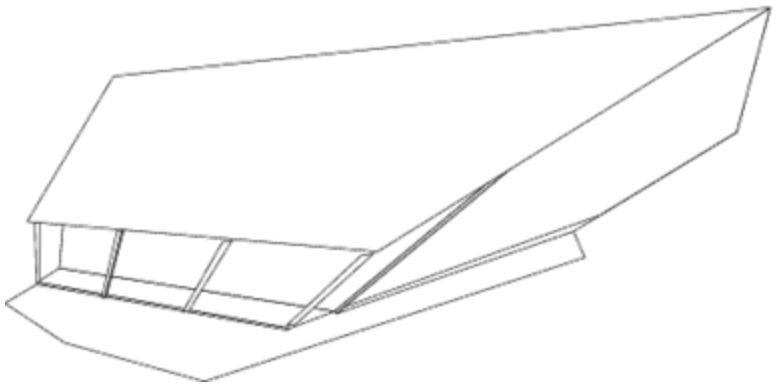
$$\mathbf{Q_p} = \begin{bmatrix} \mathrm{A} & \mathrm{t} \\ \mathrm{v}^{\mathsf{T}} & v \end{bmatrix}$$

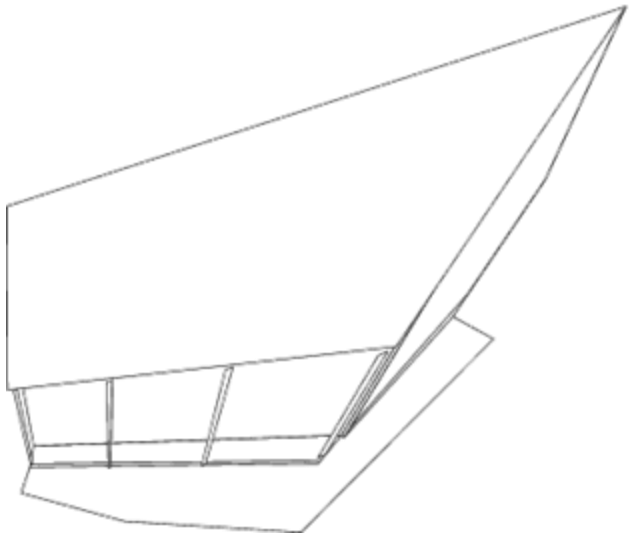
$$\mathbf{x} = \mathbf{PX} = \left(\mathbf{PQ_P^{-1}}\right)\left(\mathbf{Q_P X}\right)$$

Projective Ambiguity

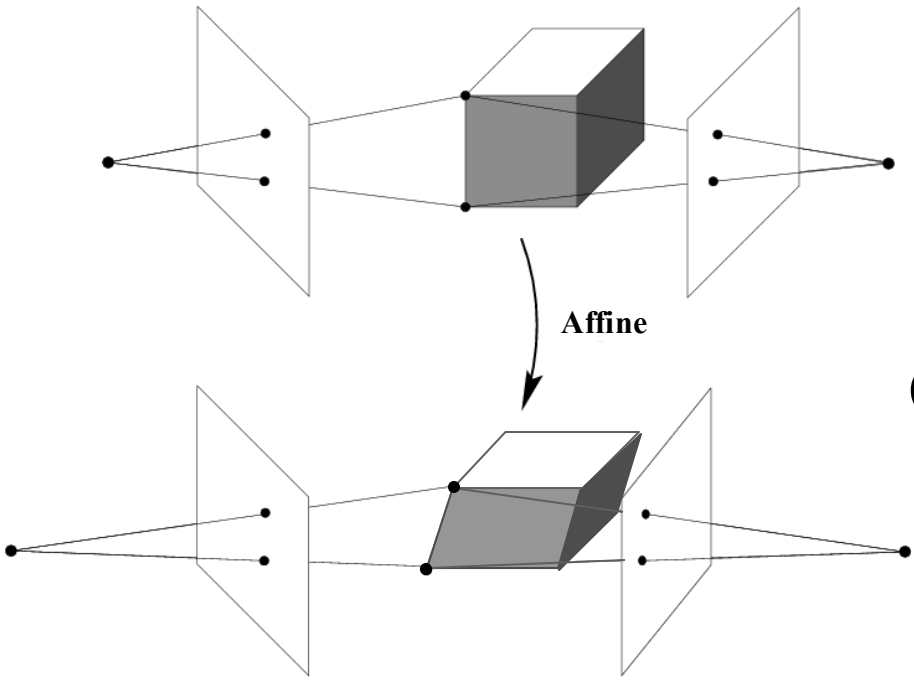








Affine Ambiguity



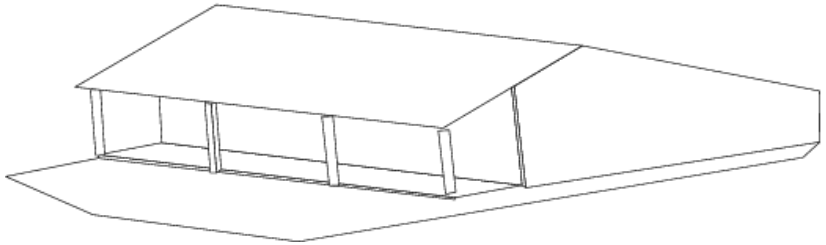
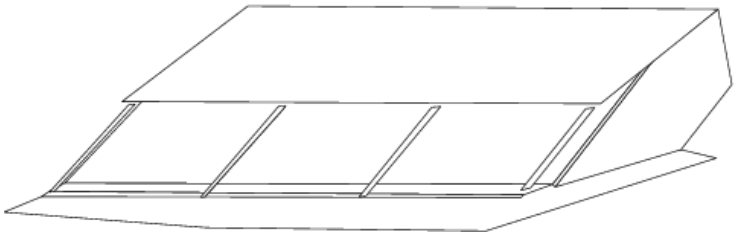
$$\mathbf{Q}_A = \begin{bmatrix} A & t \\ 0^T & 1 \end{bmatrix}$$

$$\mathbf{x} = \mathbf{PX} = \left(\mathbf{PQ}_A^{-1}\right)\left(\mathbf{Q}_A\,\mathbf{X}\right)$$

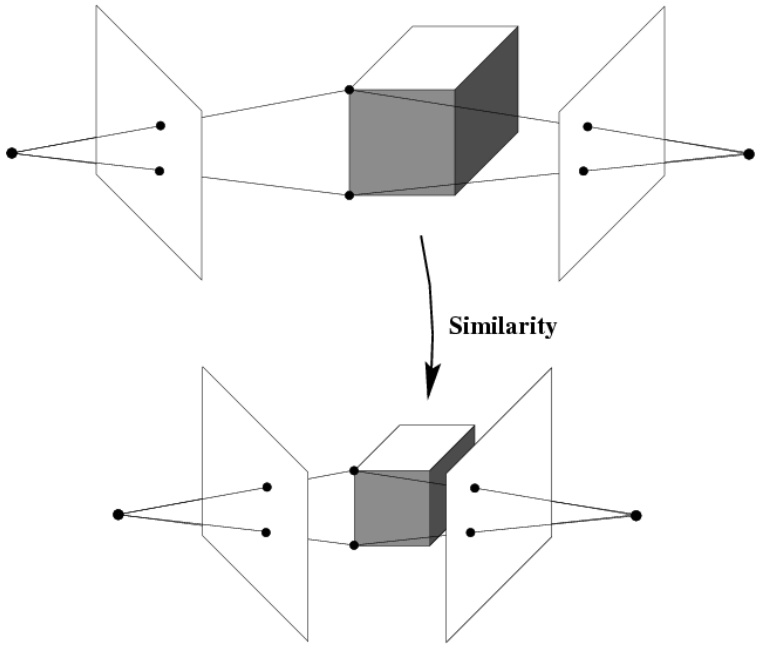
Affine Ambiguity





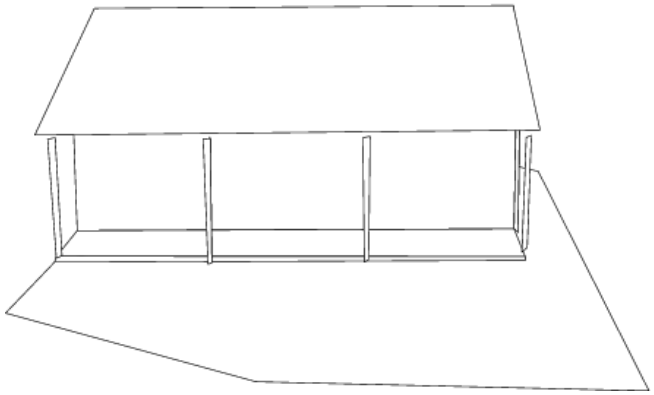
Similarity Ambiguity

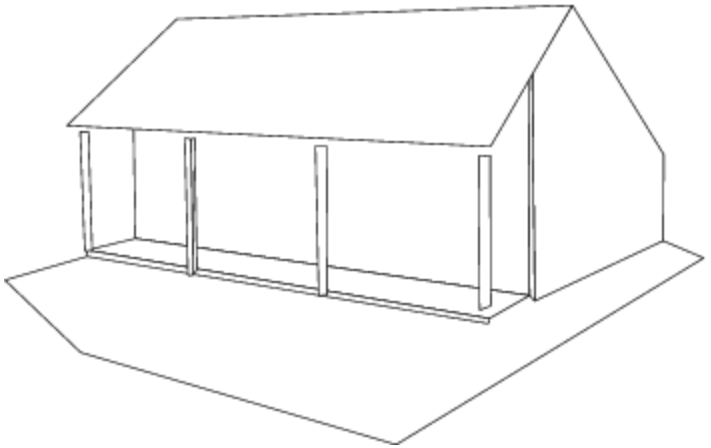


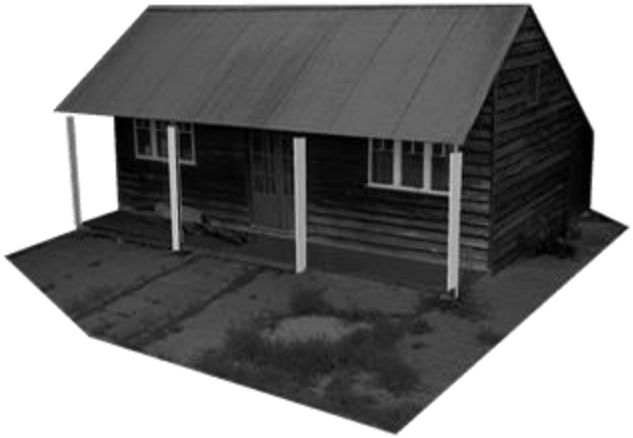
$$\mathbf{Q}_s = \begin{bmatrix} s\mathrm{R} & \mathrm{t} \\ 0^{\mathsf{T}} & 1 \end{bmatrix}$$

$$\mathbf{x} = \mathbf{P}\mathbf{X} = \left(\mathbf{P}\mathbf{Q}_S^{-1}\right)\left(\mathbf{Q}_S\mathbf{X}\right)$$

Similarity Ambiguity





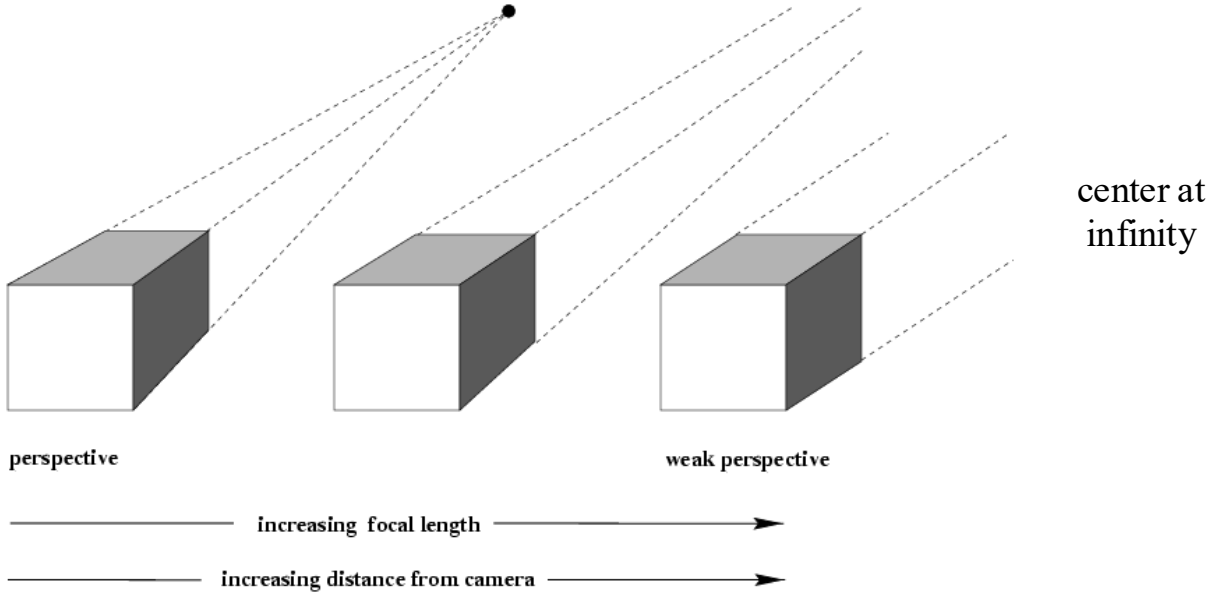




Structure from Motion: Affine Cameras

Structure from Motion

- Let's start with *affine cameras* (the math is easier)

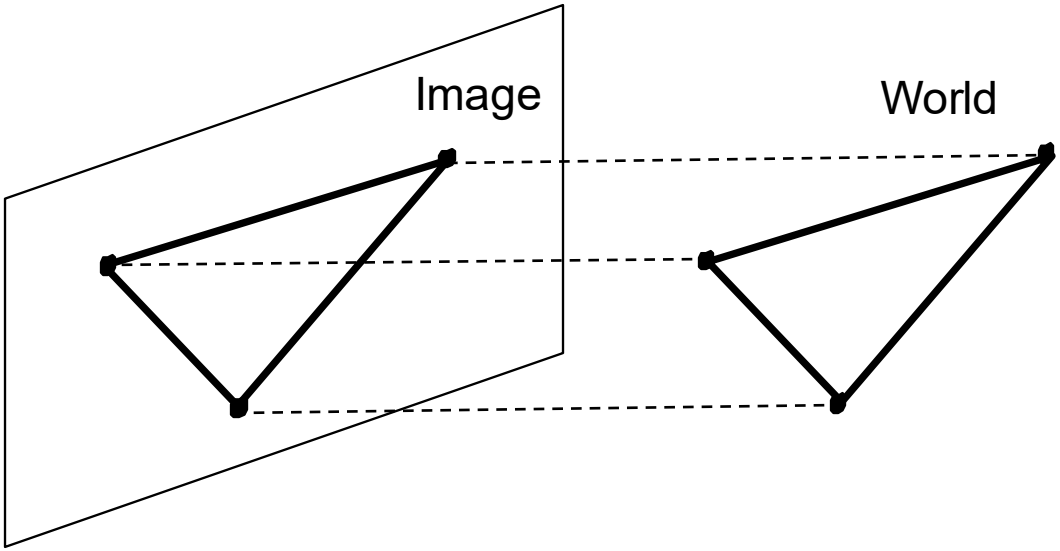






Orthographic Projection

- Special case of perspective projection
  - Distance from center of projection to image plane is infinite

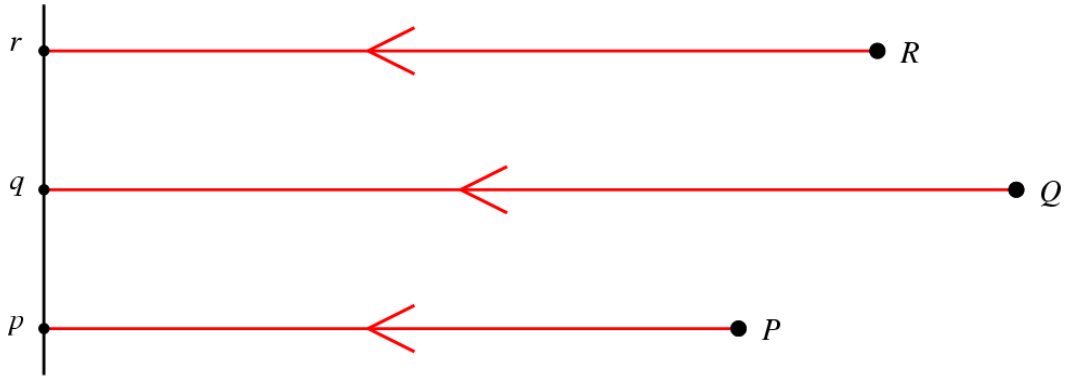


  - Projection matrix:

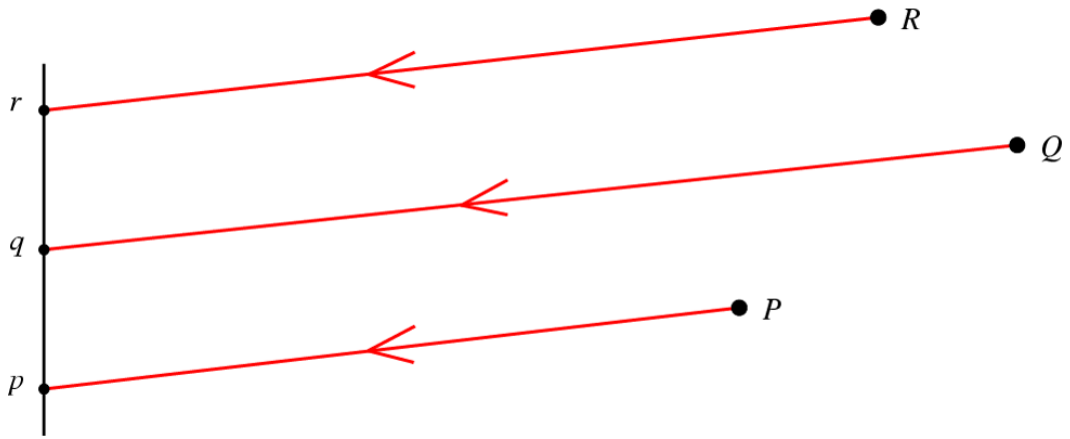
$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \Rightarrow (x, y)$$

Affine Cameras

Orthographic Projection



Parallel Projection

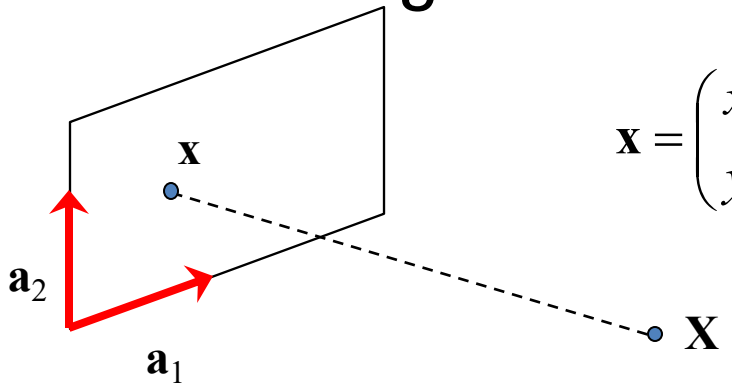


Affine Cameras

- A general affine camera combines the effects of an affine transformation of the 3D space, orthographic projection, and an affine transformation of the image:

$$\mathbf{P} = [3\times 3\,\text{affine}]\begin{bmatrix}1 & 0 & 0 & 0\\ 0 & 1 & 0 & 0\\ 0 & 0 & 0 & 1\end{bmatrix}[4\times 4\,\text{affine}] = \begin{bmatrix}a_{11} & a_{12} & a_{13} & b_1\\ a_{21} & a_{22} & a_{23} & b_2\\ 0 & 0 & 0 & 1\end{bmatrix} = \begin{bmatrix}\mathbf{A} & \mathbf{b}\\ \mathbf{0} & \mathbf{1}\end{bmatrix}$$

Affine projection is a linear mapping + translation in inhomogeneous coordinates



$$\mathbf{x} = \begin{pmatrix}x\\ y\end{pmatrix} = \begin{bmatrix}a_{11} & a_{12} & a_{13}\\ a_{21} & a_{22} & a_{23}\end{bmatrix}\begin{pmatrix}X\\ Y\\ Z\end{pmatrix} + \begin{pmatrix}b_1\\ b_2\end{pmatrix} = \mathbf{AX} + \mathbf{b}$$

Projection of world origin

Affine Structure from Motion

- Given: *m* images of *n* fixed 3D points:
  - $\mathbf{x}_{ij} = \mathbf{A}_i \mathbf{X}_j + \mathbf{b}_i, \quad i = 1, \ldots, m, \; j = 1, \ldots, n$
- Problem: use the *mn* correspondences $\mathbf{x}_{ij}$ to estimate *m* projection matrices $\mathbf{A}_i$ and translation vectors $\mathbf{b}_i$, and *n* points $\mathbf{X}_j$
- The reconstruction is defined up to an arbitrary *affine* transformation **Q** (12 degrees of freedom):

$$\begin{bmatrix} \mathbf{A} & \mathbf{b} \\ \mathbf{0} & \mathbf{1} \end{bmatrix} \rightarrow \begin{bmatrix} \mathbf{A} & \mathbf{b} \\ \mathbf{0} & \mathbf{1} \end{bmatrix} \mathbf{Q}^{-1}, \qquad \begin{pmatrix} \mathbf{X} \\ \mathbf{1} \end{pmatrix} \rightarrow \mathbf{Q} \begin{pmatrix} \mathbf{X} \\ \mathbf{1} \end{pmatrix}$$

- We have $2mn$ knowns and $8m + 3n$ unknowns (minus 12 dof for affine ambiguity)
- Thus, we must have $2mn >= 8m + 3n - 12$
- For two views, we need four point correspondences

Affine Structure from Motion

- Centering: subtract the centroid of the image points

$$\hat{\mathbf{x}}_{ij} = \mathbf{x}_{ij} - \frac{1}{n}\sum_{k=1}^{n}\mathbf{x}_{ik} = \mathbf{A}_i\mathbf{X}_j + \mathbf{b}_i - \frac{1}{n}\sum_{k=1}^{n}\left(\mathbf{A}_i\mathbf{X}_k + \mathbf{b}_i\right)$$

$$= \mathbf{A}_i\left(\mathbf{X}_j - \frac{1}{n}\sum_{k=1}^{n}\mathbf{X}_k\right) = \mathbf{A}_i\hat{\mathbf{X}}_j$$

- For simplicity, assume that the origin of the world coordinate system is at the centroid of the 3D points
- After centering, each normalized point $\mathbf{x}_{ij}$ is related to the 3D point $\mathbf{X}_i$ by

$$\hat{\mathbf{x}}_{ij} = \mathbf{A}_i\mathbf{X}_j$$

Affine Structure from Motion

- $2mn$ data (measurement) matrix:

$$\mathbf{D} = \begin{bmatrix} \hat{\mathbf{x}}_{11} & \hat{\mathbf{x}}_{12} & \cdots & \hat{\mathbf{x}}_{1n} \\ \hat{\mathbf{x}}_{21} & \hat{\mathbf{x}}_{22} & \cdots & \hat{\mathbf{x}}_{2n} \\ & & \ddots & \\ \hat{\mathbf{x}}_{m1} & \hat{\mathbf{x}}_{m2} & \cdots & \hat{\mathbf{x}}_{mn} \end{bmatrix}$$

cameras ($2\,m$)

points (n)

C. Tomasi and T. Kanade. Shape and motion from image streams under orthography: A factorization method. *IJCV*, 9(2):137-154, November 1992.

Affine Structure from Motion

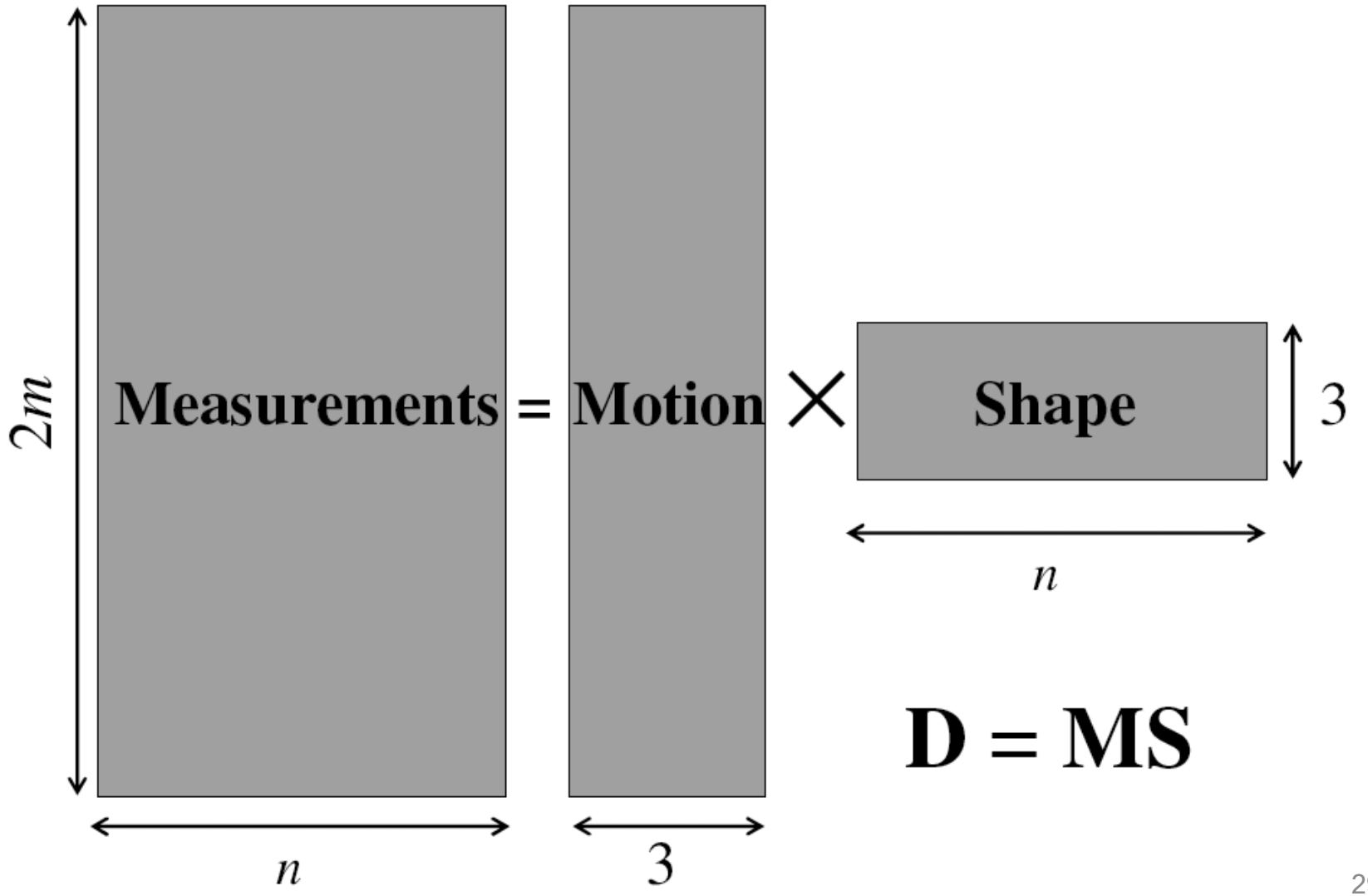
$$\mathbf{D} = \begin{bmatrix} \hat{\mathbf{x}}_{11} & \hat{\mathbf{x}}_{12} & \cdots & \hat{\mathbf{x}}_{1n} \\ \hat{\mathbf{x}}_{21} & \hat{\mathbf{x}}_{22} & \cdots & \hat{\mathbf{x}}_{2n} \\ & & \ddots & \\ \hat{\mathbf{x}}_{m1} & \hat{\mathbf{x}}_{m2} & \cdots & \hat{\mathbf{x}}_{mn} \end{bmatrix} = \begin{bmatrix} \mathbf{A}_1 \\ \mathbf{A}_2 \\ \vdots \\ \mathbf{A}_m \end{bmatrix} \begin{bmatrix} \mathbf{X}_1 & \mathbf{X}_2 & \cdots & \mathbf{X}_n \end{bmatrix}$$

points ($3 \times n$)

cameras ($2m \times 3$)

The measurement matrix $\mathbf{D} = \mathbf{MS}$ must have rank 3!

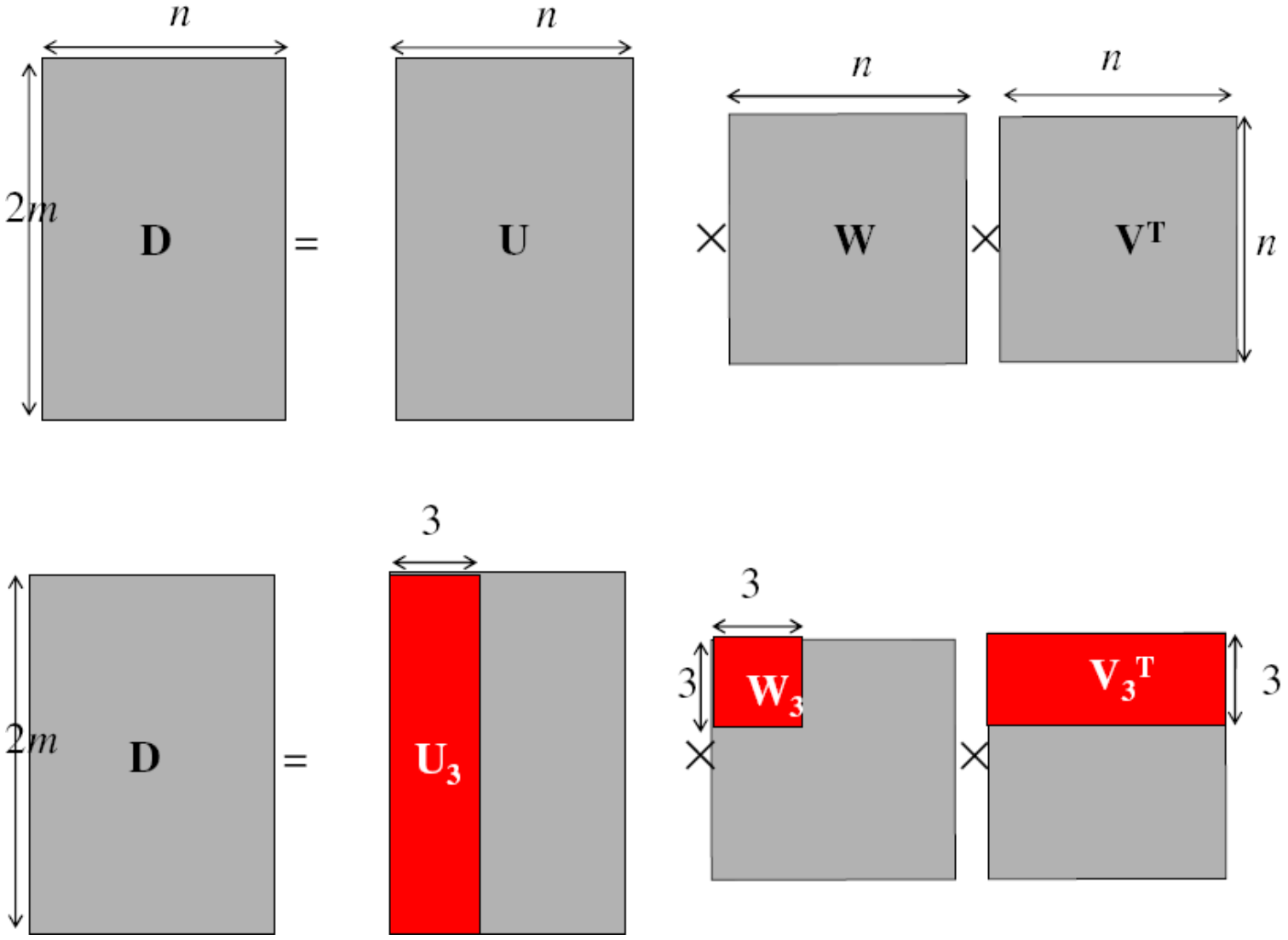
Factorizing the Measurement Matrix



$$\mathbf{D} = \mathbf{MS}$$

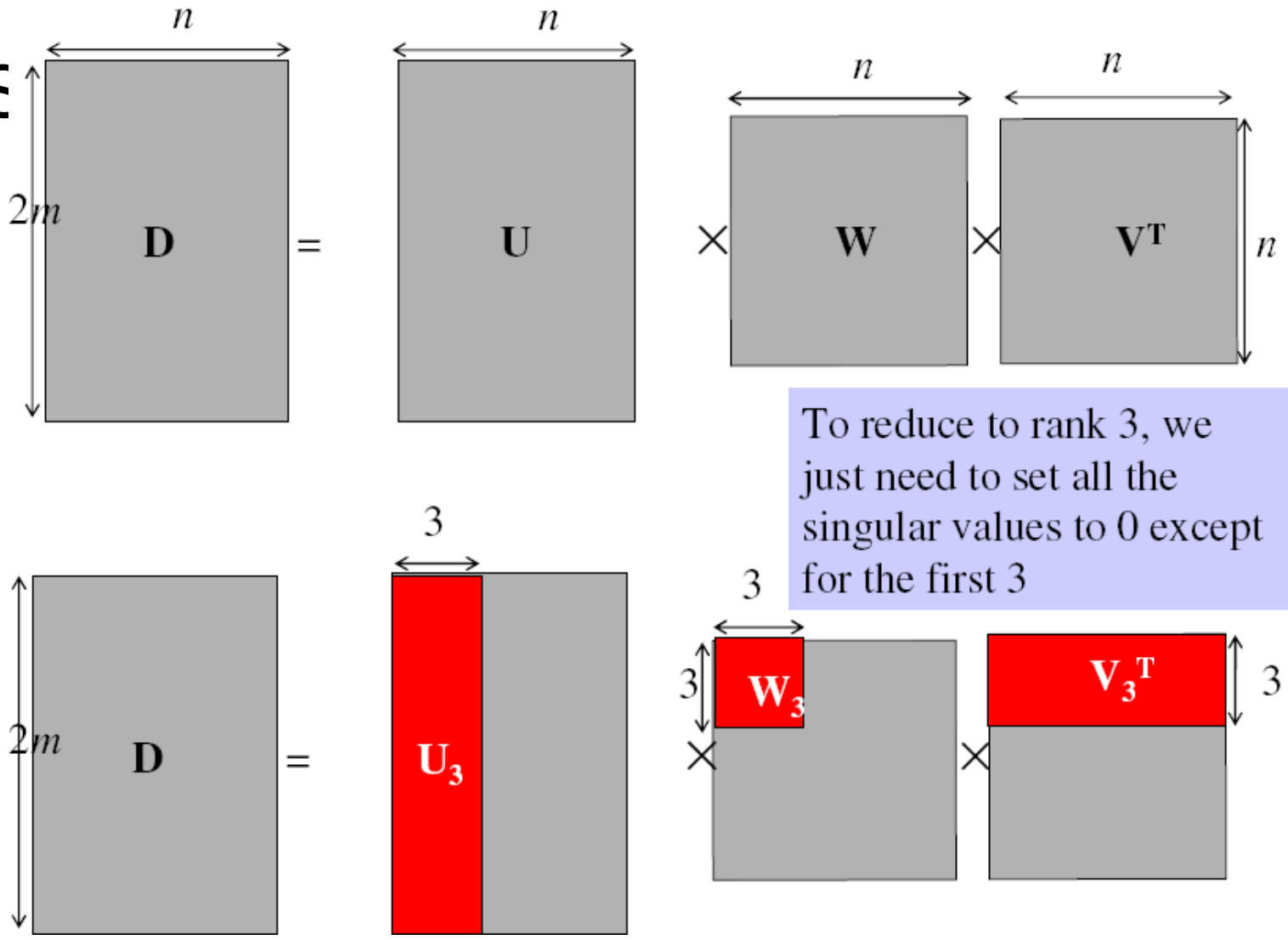
Factorizing the Measurement Matrix

- Singular value decomposition of D:

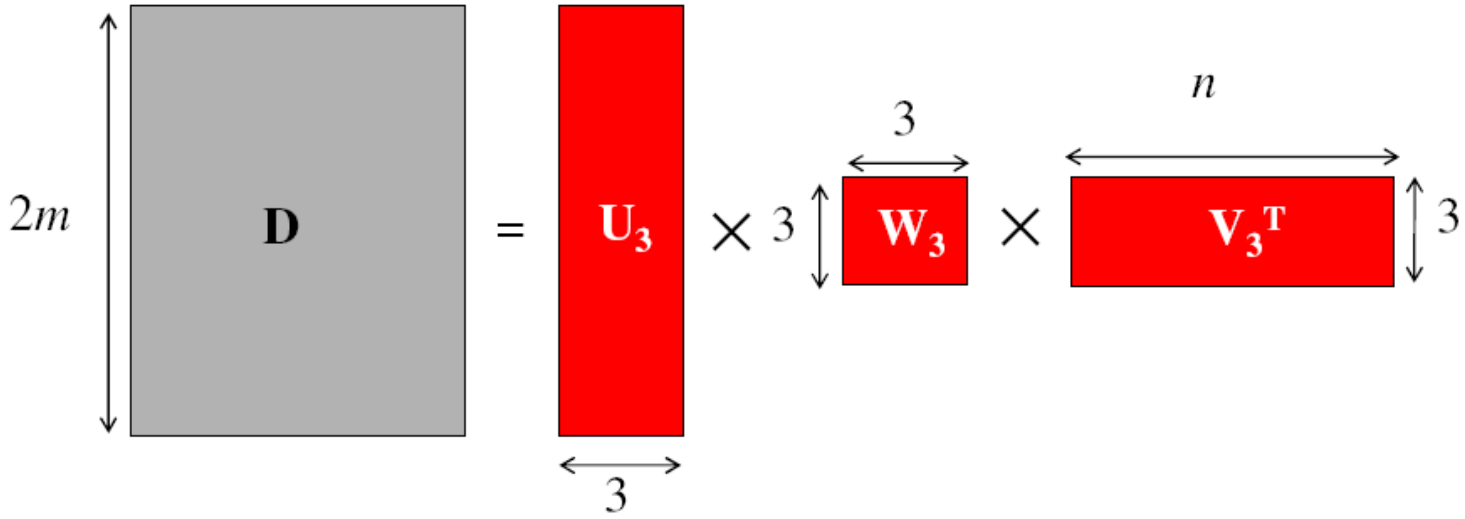


Factorizing the Measurement Matrix

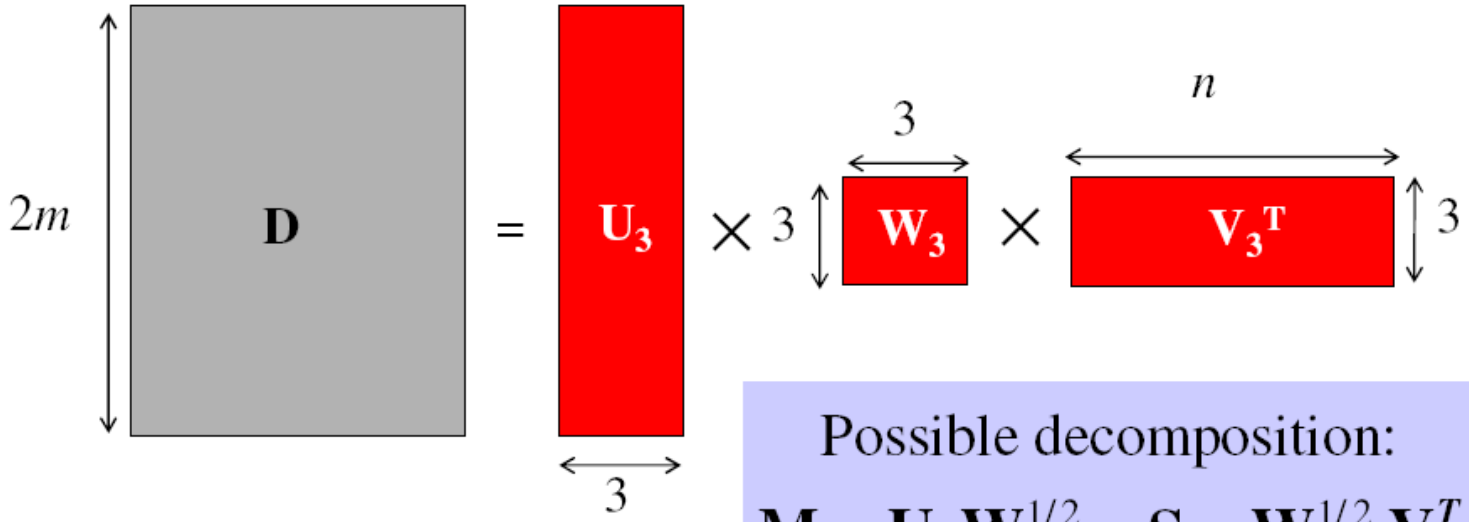
- S



Factorizing the Measurement Matrix

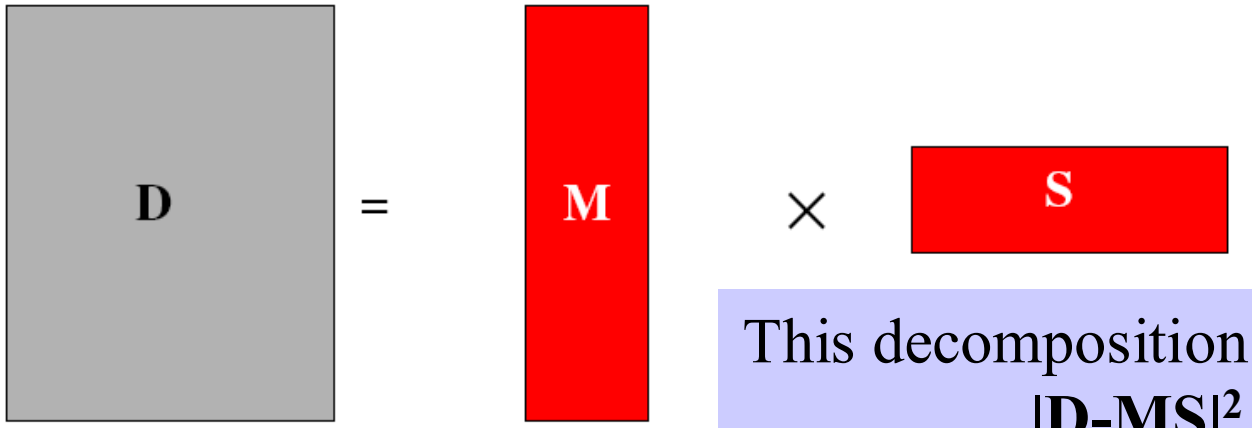


Factorizing the Measurement Matrix



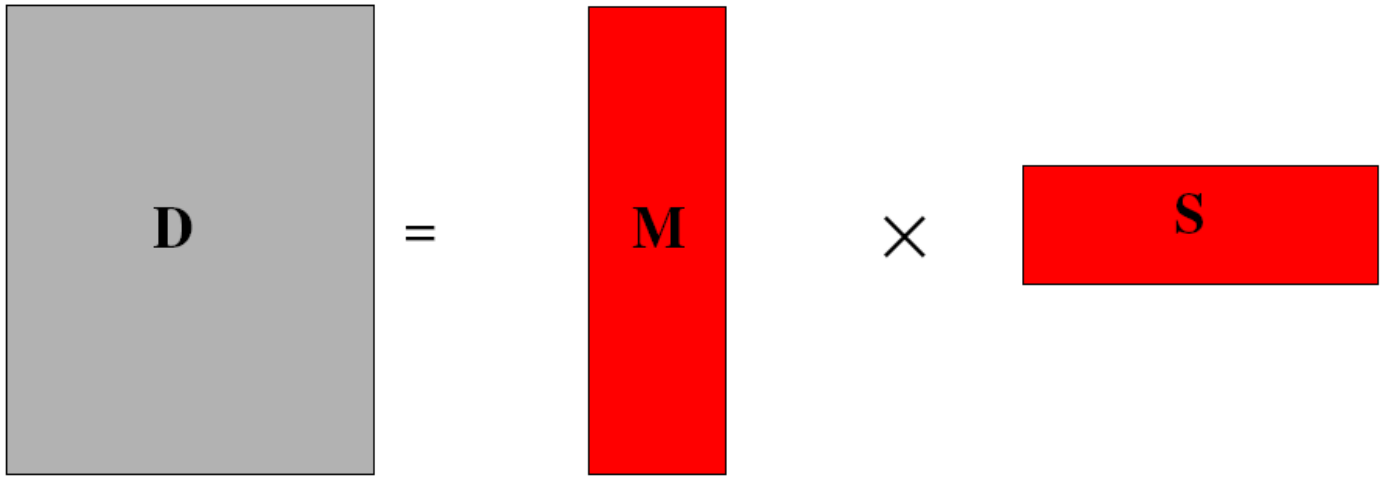
Possible decomposition:

$$\mathbf{M} = \mathbf{U}_3 \mathbf{W}_3^{1/2} \qquad \mathbf{S} = \mathbf{W}_3^{1/2} \mathbf{V}_3^T$$



This decomposition minimizes $|\mathbf{D}-\mathbf{MS}|^2$

Affine Ambiguity



- The decomposition is not unique. We get the same **D** by using any 3×3 matrix **C** and applying the transformations $\mathbf{M} \rightarrow \mathbf{MC}$, $\mathbf{S} \rightarrow \mathbf{C}^{-1}\mathbf{S}$
- That is because we have only an affine transformation and we have not enforced any Euclidean constraints (like forcing the image axes to be perpendicular, for example)

Algorithm Summary

- Given: m images and n features $\mathbf{x}_{ij}$
- For each image i, center the feature coordinates
- Construct a $2m \times n$ measurement matrix $\mathbf{D}$:
  - Column j contains the projection of point j in all views
  - Row i contains one coordinate of the projections of all the n points in image i
- Factorize $\mathbf{D}$:
  - Compute SVD: $\mathbf{D} = \mathbf{U}\,\mathbf{W}\,\mathbf{V}^T$
  - Create $\mathbf{U}_3$ by taking the first 3 columns of $\mathbf{U}$
  - Create $\mathbf{V}_3$ by taking the first 3 columns of $\mathbf{V}$
  - Create $\mathbf{W}_3$ by taking the upper left 3×3 block of $\mathbf{W}$
- Create the motion and shape matrices:
  - $\mathbf{M} = \mathbf{U}_3\mathbf{W}_3^{½}$ and $\mathbf{S} = \mathbf{W}_3^{½}\,\mathbf{V}_3^T$ (**or** $\mathbf{M} = \mathbf{U}_3$ and $\mathbf{S} = \mathbf{W}_3\mathbf{V}_3^T$)
- Eliminate affine ambiguity

Eliminating the affine ambiguity

- So far, we have obtained one solution:

$$\boldsymbol{D} = \underbrace{\begin{bmatrix} \boldsymbol{A}_1 \\ \boldsymbol{A}_2 \\ \vdots \\ \boldsymbol{A}_m \end{bmatrix}}_{2m\times 3} \underbrace{[\boldsymbol{X}_1 \quad \boldsymbol{X}_2 \quad \cdots \quad \boldsymbol{X}_n]}_{3\times n}$$

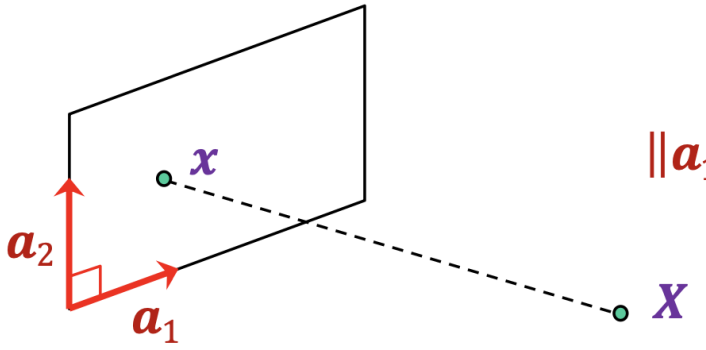
- We want:

$$\boldsymbol{D} = \begin{bmatrix} \boldsymbol{A}_1\boldsymbol{Q} \\ \boldsymbol{A}_2\boldsymbol{Q} \\ \vdots \\ \boldsymbol{A}_m\boldsymbol{Q} \end{bmatrix} [\boldsymbol{Q}^{-1}\boldsymbol{X}_1 \quad \boldsymbol{Q}^{-1}\boldsymbol{X}_2 \quad \cdots \quad \boldsymbol{Q}^{-1}\boldsymbol{X}_n]$$

such that each camera matrix $\boldsymbol{A}_i\boldsymbol{Q}$ represents orthographic projection, i.e., has orthonormal axes (rows)

Eliminating the affine ambiguity

- Let $\boldsymbol{a}_1$ and $\boldsymbol{a}_2$ be the rows of a 2×3 orthographic projection matrix. Then



$$\boldsymbol{a}_1 \cdot \boldsymbol{a}_2 = 0$$
$$\|\boldsymbol{a}_1\|^2 = \|\boldsymbol{a}_2\|^2 = 1$$

- This translates into $3m$ constraints on the 9 entries of $\boldsymbol{Q}$:

$$(\boldsymbol{A}_i\boldsymbol{Q})(\boldsymbol{A}_i\boldsymbol{Q})^T = \boldsymbol{A}_i(\boldsymbol{Q}\boldsymbol{Q}^T)\boldsymbol{A}_i^T = \mathbf{I}_{2\times 2}, \qquad i = 1, \ldots, m$$

  - Are the constraints linear?
  - First, solve for $\boldsymbol{L} = \boldsymbol{Q}\boldsymbol{Q}^T$
  - Recover $\boldsymbol{Q}$ from $\boldsymbol{L}$ by Cholesky decomposition
  - Update $\boldsymbol{M}$ to $\boldsymbol{M}\boldsymbol{Q}$, $\boldsymbol{S}$ to $\boldsymbol{Q}^{-1}\boldsymbol{S}$

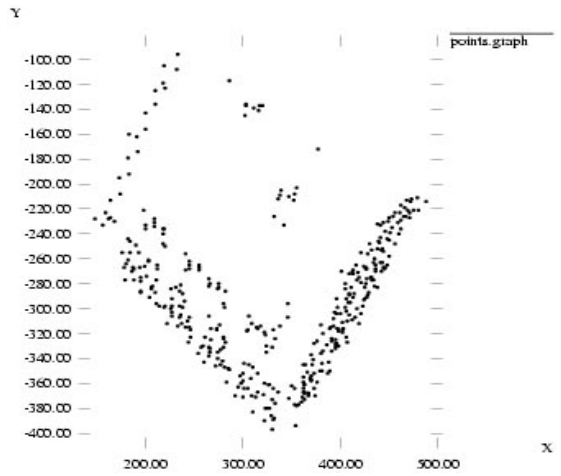
Reconstruction Results



1



60

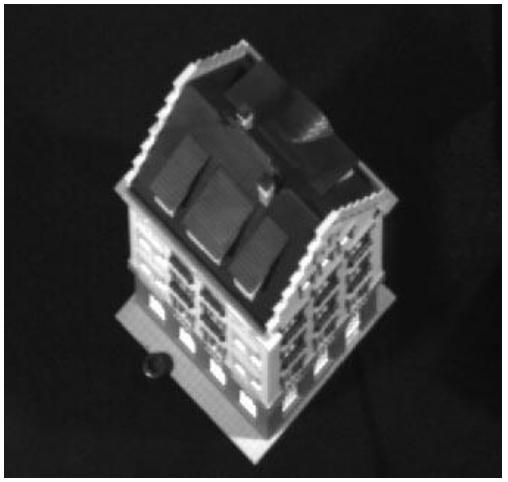




120



150

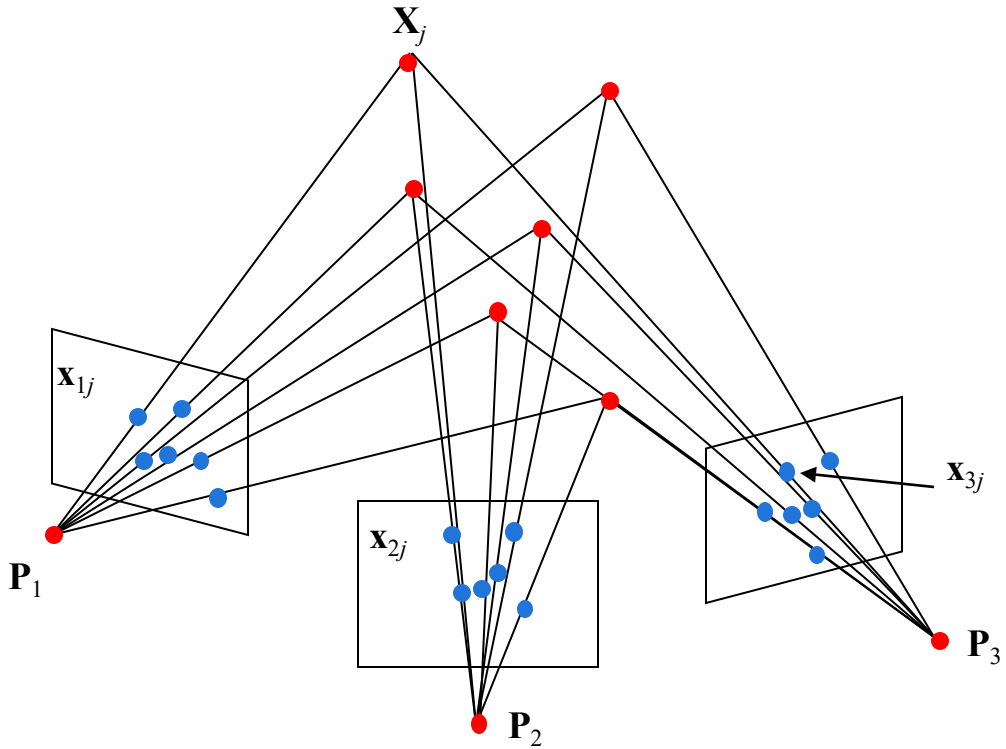


C. Tomasi and T. Kanade. Shape and motion from image streams under orthography: A factorization method. *IJCV*, 9(2):137-154, November 1992.

Structure from Motion: Perspective Cameras

Projective Structure from Motion

- Given: *m* images of *n* fixed 3D points
  - $\mathbf{x}_{ij} = \mathbf{P}_i \mathbf{X}_j, \qquad i = 1, \ldots, m, \quad j = 1, \ldots, n$
- Problem: estimate *m* projection matrices $\mathbf{P}_i$ and *n* 3D points $\mathbf{X}_j$ from the *mn* correspondences $\mathbf{x}_{ij}$



Projective Structure from Motion

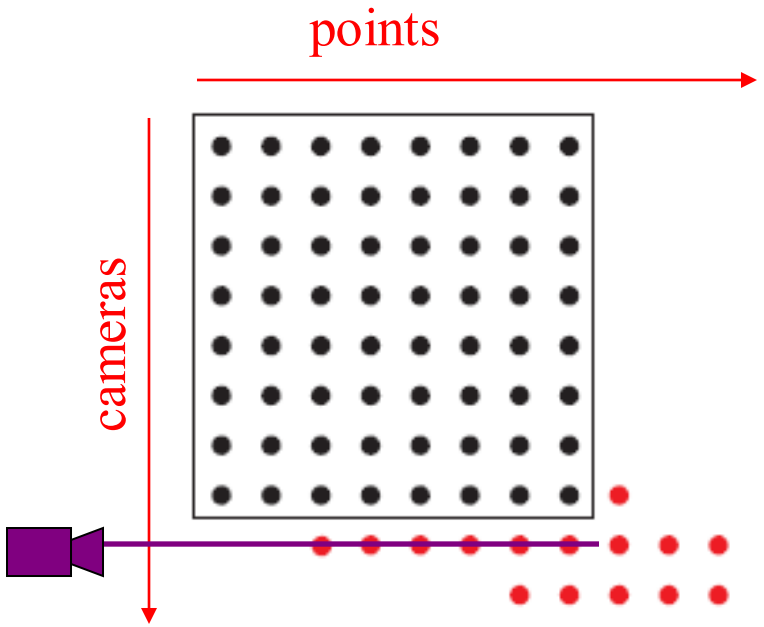
- Given: *m* images of *n* fixed 3D points
  - $z_{ij}\,\mathbf{x}_{ij} = \mathbf{P}_i\,\mathbf{X}_j\,, \qquad i = 1, \ldots, m, \quad j = 1, \ldots, n$
- Problem: estimate *m* projection matrices $\mathbf{P}_i$ and *n* 3D points $\mathbf{X}_j$ from the *mn* correspondences $\mathbf{x}_{ij}$
- With no calibration info, cameras and points can only be recovered up to a 4x4 projective transformation **Q**:
  - $\mathbf{X} \rightarrow \mathbf{QX}$, $\mathbf{P} \rightarrow \mathbf{PQ}^{-1}$
- We can solve for structure and motion when
  - $2mn >= 11m + 3n - 15$
- For two cameras, at least 7 points are needed

Projective SFM: Two-camera Case

- Compute fundamental matrix $\mathbf{F}$ between the two views
- First camera matrix: $[\mathbf{I}|\mathbf{0}]$
- Second camera matrix: $[\mathbf{A}|\mathbf{b}]$
- Then $\mathbf{b}$ is the epipole ($\mathbf{F}^{\mathrm{T}}\mathbf{b} = 0$), $\mathbf{A} = -[\mathbf{b}_{\times}]\mathbf{F}$

Sequential Structure from Motion

- Initialize motion from two images using fundamental matrix
- Initialize structure by triangulation
- For each additional view:
  - Determine projection matrix of new camera using all the known 3D points that are visible in its image – *calibration*



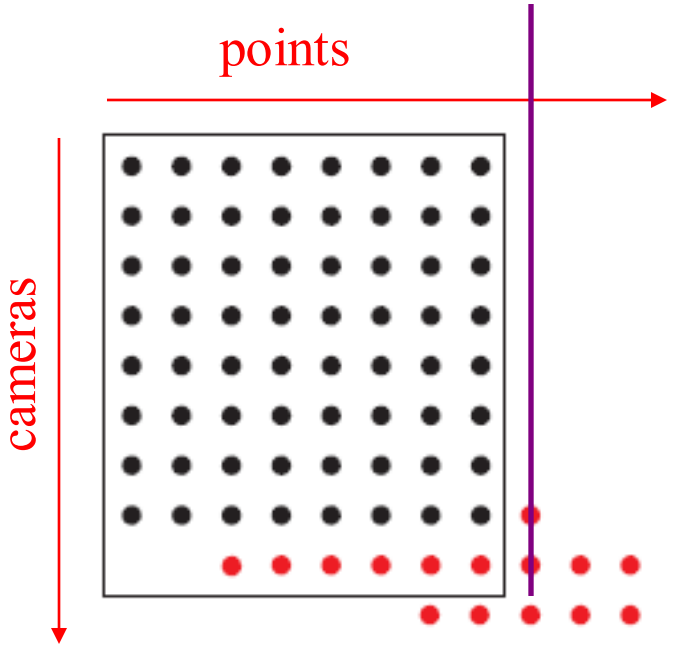
Sequential Structure from Motion

•Initialize motion from two images using fundamental matrix

•Initialize structure by triangulation

•For each additional view:

- Determine projection matrix of new camera using all the known 3D points that are visible in its image – *calibration*
- Refine and extend structure: compute new 3D points, re-optimize existing points that are also seen by this camera – *triangulation*



Sequential Structure from motion

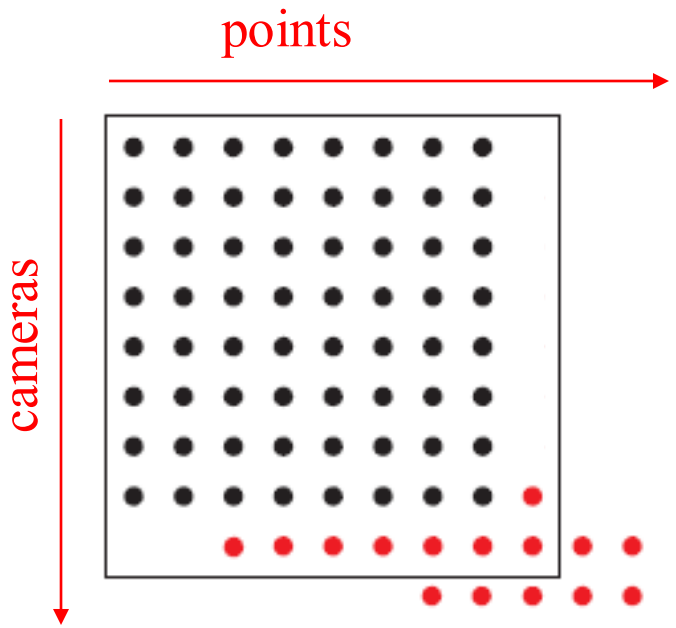
•Initialize motion from two images using fundamental matrix

•Initialize structure by triangulation

•For each additional view:

- Determine projection matrix of new camera using all the known 3D points that are visible in its image – *calibration*
- Refine and extend structure: compute new 3D points, re-optimize existing points that are also seen by this camera – *triangulation*

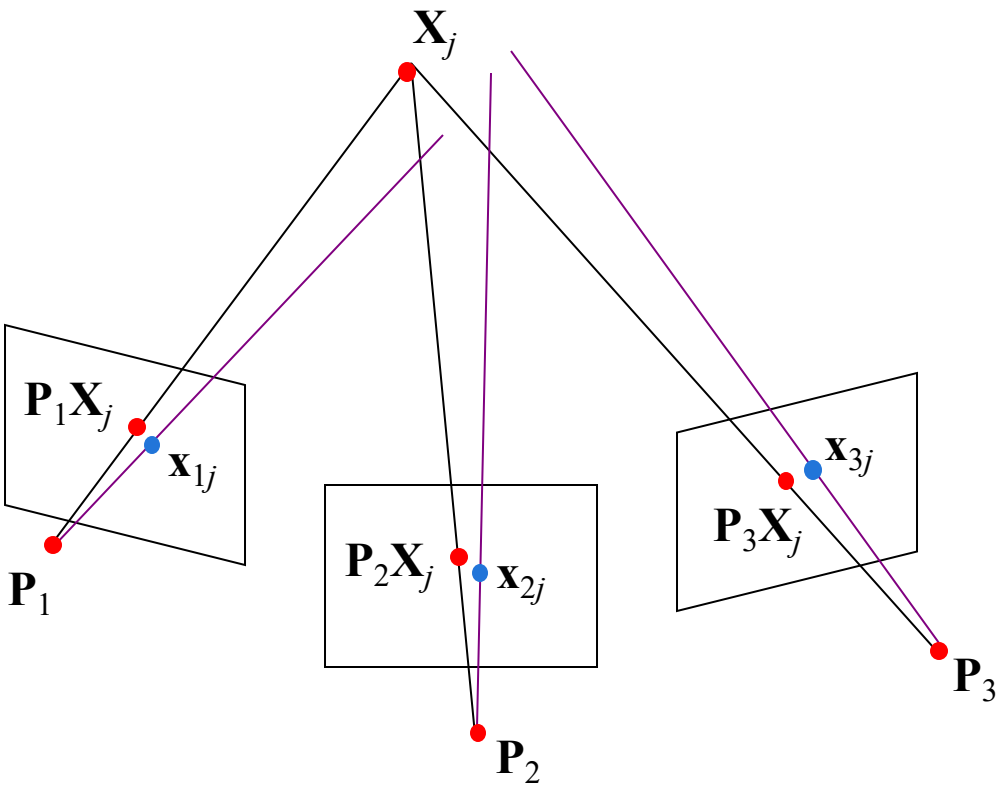
•Refine structure and motion: bundle adjustment



Bundle Adjustment

- Non-linear method for refining structure and motion
- Minimizing reprojection error

$$E(\mathbf{P},\mathbf{X}) = \sum_{i=1}^{m}\sum_{j=1}^{n} D\left(\mathbf{x}_{ij}, \mathbf{P}_i\mathbf{X}_j\right)^2$$



Self-calibration

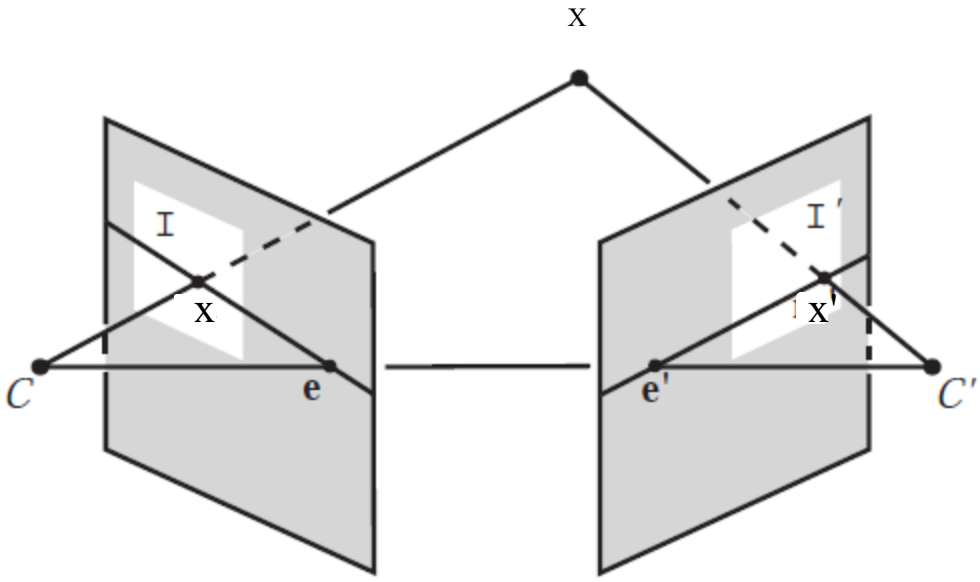
- Self-calibration (auto-calibration) is the process of determining intrinsic camera parameters directly from uncalibrated images
- For example, when the images are acquired by a single moving camera, we can use the constraint that the intrinsic parameter matrix remains fixed for all the images
  - Compute initial projective reconstruction and find 3D projective transformation matrix $\mathbf{Q}$ such that all camera matrices are in the form $\mathbf{P}_i = \mathbf{K}\,[\mathbf{R}_i \mid \mathbf{t}_i]$
- Can use constraints on the form of the calibration matrix: zero skew
- Can use vanishing points

Triangulation

More formulations exist

Triangulation: Linear Solution

- Generally, rays C→x and C'→x' will not exactly intersect
- Can solve via SVD, finding a least squares solution to a system of equations



$$\mathbf{x} = \mathbf{PX} \qquad \mathbf{x}' = \mathbf{P}'\mathbf{X}$$

$$\mathbf{AX} = \mathbf{0} \quad \mathbf{A} = \begin{bmatrix} u\mathbf{p}_3^T - \mathbf{p}_1^T \\ v\mathbf{p}_3^T - \mathbf{p}_2^T \\ u'\mathbf{p}_3'^T - \mathbf{p}_1'^T \\ v'\mathbf{p}_3'^T - \mathbf{p}_2'^T \end{bmatrix}$$

From $\mathbf{x} \times \mathbf{PX} = 0$ and $\mathbf{x}' \times \mathbf{PX}' = 0$

Triangulation: Linear Solution

Given **P**, **P'** , **x**, **x'**

1. Precondition points and projection matrices
2. Create matrix **A**
3. [U, S, V] = svd(A)
4. **X** = V(:, end)

$$\mathbf{x} = w\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} \qquad \mathbf{x}' = w\begin{bmatrix} u' \\ v' \\ 1 \end{bmatrix}$$

$$\mathbf{P} = \begin{bmatrix} \mathbf{p}_1^T \\ \mathbf{p}_2^T \\ \mathbf{p}_3^T \end{bmatrix} \qquad \mathbf{P}' = \begin{bmatrix} \mathbf{p}_1'^T \\ \mathbf{p}_2'^T \\ \mathbf{p}_3'^T \end{bmatrix}$$

Pros and Cons

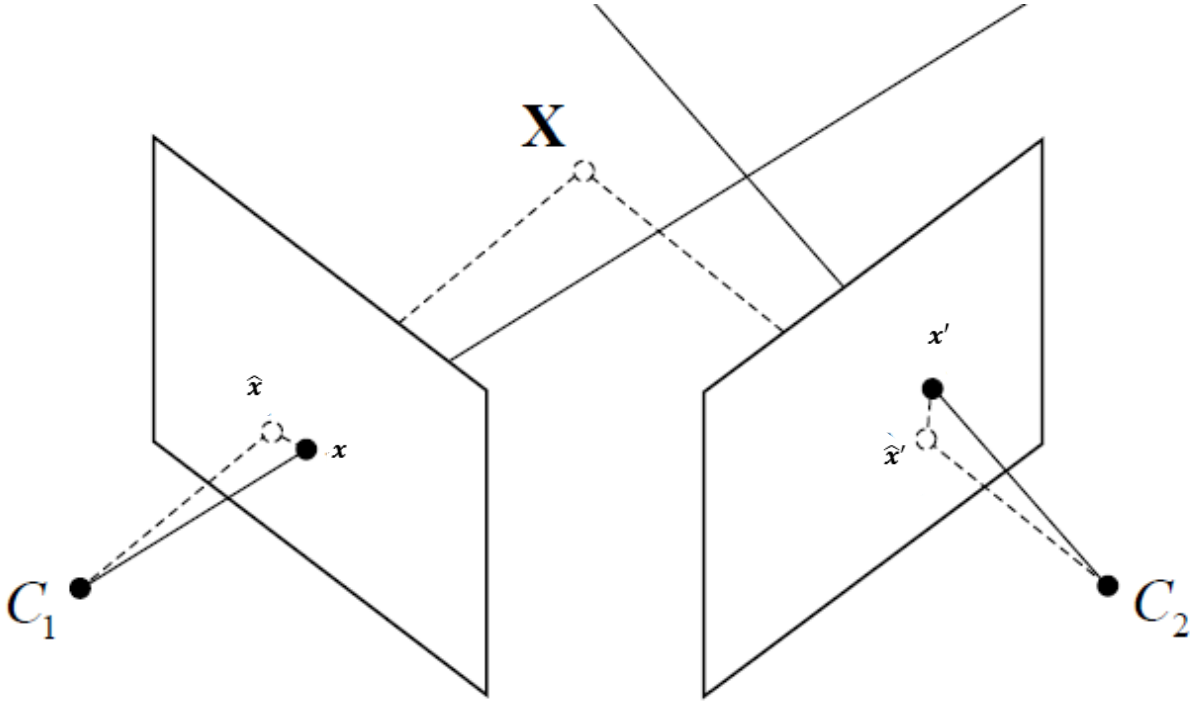
- Works for any number of corresponding images
- Not projectively invariant

$$\mathbf{A} = \begin{bmatrix} u\mathbf{p}_3^T - \mathbf{p}_1^T \\ v\mathbf{p}_3^T - \mathbf{p}_2^T \\ u'\mathbf{p}_3'^T - \mathbf{p}_1'^T \\ v'\mathbf{p}_3'^T - \mathbf{p}_2'^T \end{bmatrix}$$

Triangulation: Non-linear Solution

- Minimize projected error while satisfying $\hat{\boldsymbol{x}}'^{T}\boldsymbol{F}\hat{\boldsymbol{x}}=0$

$$cost(\boldsymbol{X}) = dist(\boldsymbol{x}, \hat{\boldsymbol{x}})^2 + dist(\boldsymbol{x}', \hat{\boldsymbol{x}}')^2$$



Triangulation: Non-linear Solution

- Minimize projected error while satisfying $\hat{\boldsymbol{x}}'^{T}\boldsymbol{F}\hat{\boldsymbol{x}}=0$

$$cost(\boldsymbol{X}) = dist(\boldsymbol{x}, \hat{\boldsymbol{x}})^2 + dist(\boldsymbol{x}', \hat{\boldsymbol{x}}')^2$$

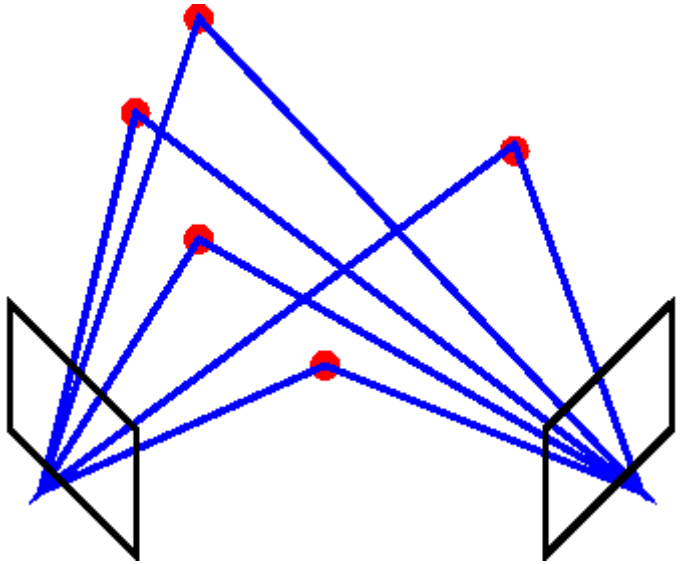


- Solution is a 6-degree polynomial of *t*, minimizing $d(\mathbf{x}, \mathbf{l}(t))^2 + d(\mathbf{x}', \mathbf{l}'(t))^2$

Camera Pose Estimation

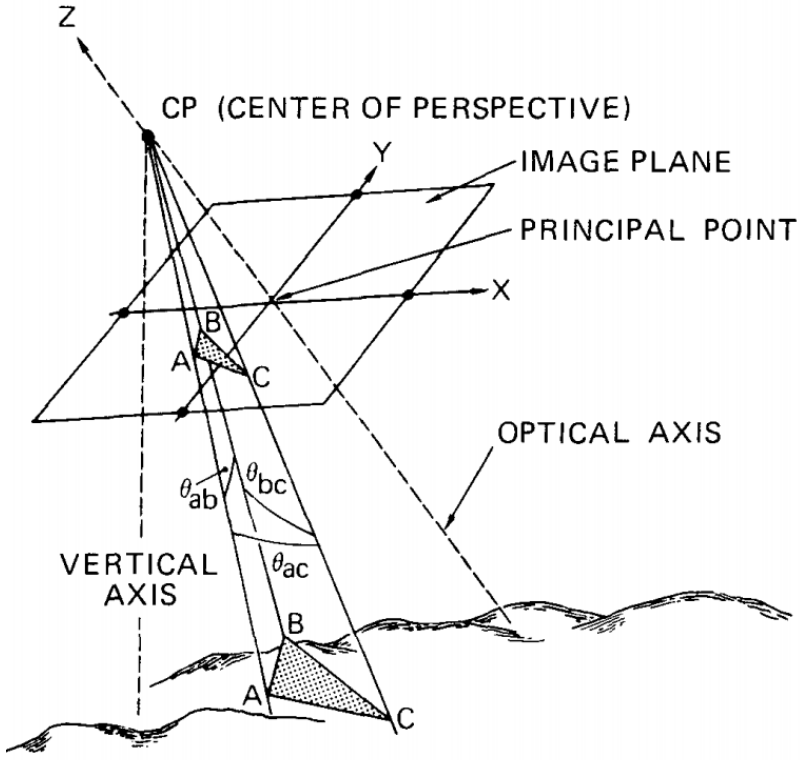
The 5-point Algorithm

- Computes relative camera pose given a minimal number of 5 correspondences
  - Up to 10 real solutions
- Intrinsics must be known
- Code available at http://vis.uky.edu/~stewe/FIVEPOINT/



The 3-point Algorithm

- A.k.a. the Perspective 3 Point problem (Haralick et al. 2004)
- Estimate camera pose given images of three known 3D points
  - Up to 4 real solutions



Bundle Adjustment

Bundle Adjustment

- Refines a visual reconstruction to produce jointly optimal 3D structure and viewing parameters

- *'Bundle'* refers to the bundle of light rays leaving each 3D feature and converging on each camera center.

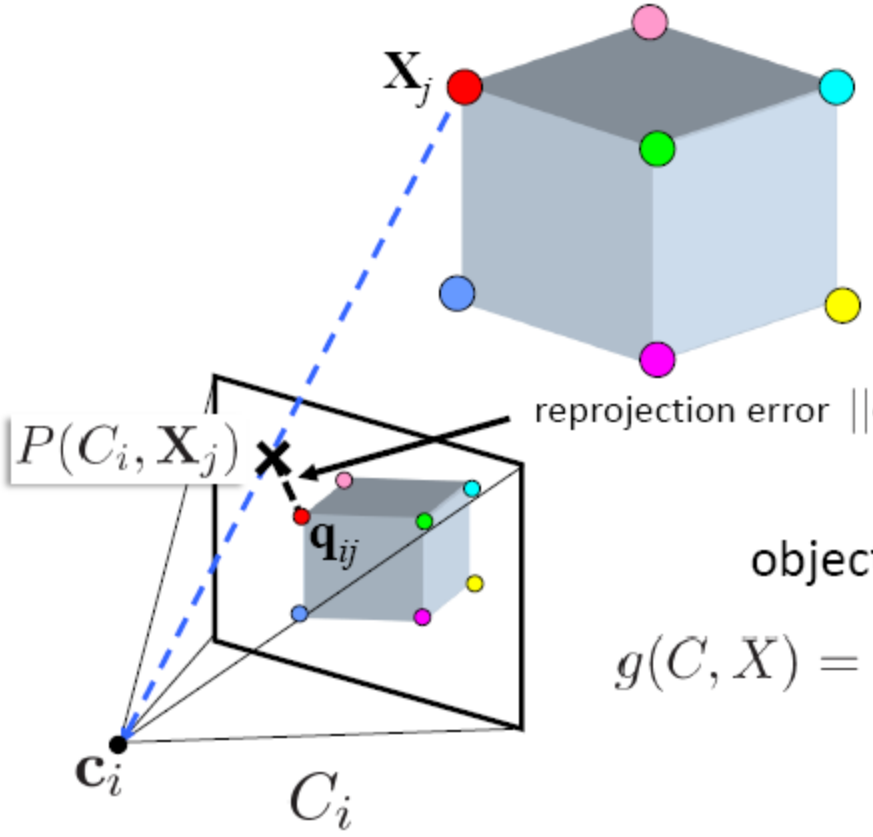
Bundle Adjustment

Bundle Adjustment

- Refines a visual reconstruction to produce jointly optimal 3D structure and viewing parameters

- *'Bundle'* refers to the bundle of light rays leaving each 3D feature and converging on each camera center.

Reprojection Error



reprojection error $||\mathbf{q}_{ij} - P(C_i, \mathbf{X}_j)||$

objective function:

$$g(C, X) = \sum_{i=1}^{n} \sum_{j=1}^{m} w_{ij} ||\mathbf{q}_{ij} - P(C_i, \mathbf{X}_j)||^2$$

indicator variable:
1 if point *j* is visible in camera *i*
0 otherwise

Notation

- Structure and Cameras being parameterized by a single large vector **x**
- Small displacement in **x** represented by $\partial \boldsymbol{x}$
- Observations denoted by $\underline{z}$
- Predicted values at parameter value **x**, denoted by z = z(x)
- Residual prediction error, $\Delta z(\boldsymbol{x}) = \underline{z} - z(\boldsymbol{x})$

Objective Function

- Minimization of weighted sum of squared error (SSE) cost function:

$$f(x) \equiv \frac{1}{2} \sum_i \triangle z_i(x)^T W_i \triangle z_i(x) , \qquad \triangle z_i(x) \equiv \underline{z}_i - z_i(x)$$

Optimization Techniques

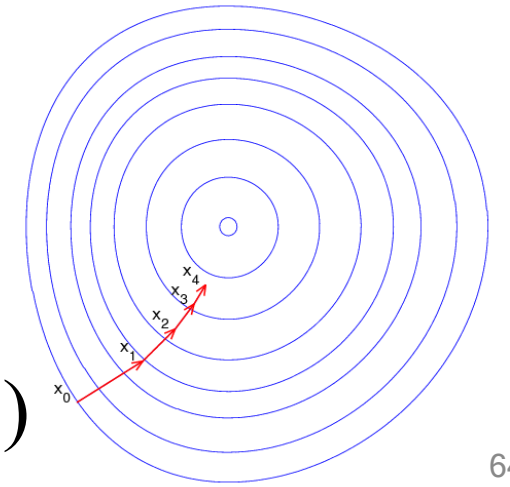
- Gradient Descent Method
- Newton-Raphson Method
- Gauss – Newton Method
- Levenberg – Marquardt Method

Gradient Descent Method

- A first-order optimization algorithm.
- To find a local minimum of a function using gradient descent, one takes steps proportional to the *negative* of the gradient of the function at the current point.
- It is robust when x is far from optimum but has poor final convergence

While $k<k_{max}$

$$x_k = x_{k-1} - \lambda \nabla f(x_{k-1})$$



Newton - Raphson Method

- Second order optimization method
- Newton's method can often converge remarkably quickly, especially if the iteration begins "sufficiently near" the solution

$$f(x+\delta x) \approx f(x) + g^{\top}\delta x + \tfrac{1}{2}\delta x^{\top} H\,\delta x$$

quadratic local model

$$g \equiv \frac{df}{dx}(x)$$

gradient vector

$$H \equiv \frac{d^2 f}{dx^2}(x)$$

Hessian matrix

$$\frac{df}{\delta x}(x+\delta x) \approx H\,\delta x + g$$

$$\delta x = -H^{-1} g$$

Newton – Raphson Method

- For a quadratic function it converges in one iteration
- For other general function, its asymptotic convergence is quadratic

- The disadvantage of this method is the high computation complexity of H^{-1}

Gauss - Newton Method

- The Gauss-Newton algorithm is a method used to solve non-linear least squares problems
- Uses Jacobian Matrix to approximate the Hessian Matrix
- The gradient is the vector formed by the partial derivatives of a *scalar* function
- The Jacobian matrix is the matrix formed by the partial derivatives of a *vector* function. Its vectors are the gradients of the respective components of the function.

Gauss - Newton Method

- The Gauss–Newton algorithm is a method used to solve non-linear least squares problems

$$J \equiv \frac{dz}{dx}$$

$$f(x) \equiv \tfrac{1}{2} \triangle z(x)^{\top} W \triangle z(x) \qquad \triangle z_i(x) \equiv \underline{z}_i - z_i(x) \qquad \mathbf{J} = \begin{bmatrix} \frac{\partial \mathbf{f}}{\partial x_1} & \cdots & \frac{\partial \mathbf{f}}{\partial x_n} \end{bmatrix} = \begin{bmatrix} \nabla^{\mathrm{T}} f_1 \\ \vdots \\ \nabla^{\mathrm{T}} f_m \end{bmatrix} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \cdots & \frac{\partial f_m}{\partial x_n} \end{bmatrix}$$

$$g \equiv \frac{df}{dx} = \triangle z^{\top} W J \qquad H \equiv \frac{d^2 f}{dx^2} = J^{\top} W J + \sum_i (\triangle z^{\top} W)_i \frac{d^2 z_i}{dx^2}$$

$$H \approx J^{\top} W J. \qquad \frac{d^2 z_i}{dx^2} \approx 0$$

Normal Equations $\quad (J^{\top} W J)\, \delta x = -J^{\top} W \triangle z$

Gauss - Newton Method

- For well-parameterized bundle problems under an outlier-free least squares cost model evaluated near the cost minimum, the Gauss-Newton approximation is usually very accurate

Levenberg – Marquardt Algorithm

- LMA interpolates between the Gauss Newton algorithm (GNA) and gradient descent
- When it is far from the minimum it acts as a steepest descent and it performs Gauss Newton iterations when it is near the solution

$$(\mathsf{H} + \lambda \mathsf{W})\, \delta \mathsf{x} = -\mathsf{g}$$

$$\lambda \gg 1 \Rightarrow Gradient\ Descent\ Method$$
$$\lambda < 1 \Rightarrow Gauss - Newton\ Method$$

General Implementation Issues

- Exploit the problem structure
  - See reduced camera matrices below
- Use factorization effectively
- Use stable local parameterizations
- Scaling and preconditioning



SfM in large areas, limited image overlap



Highly overlapping images in small area

Additional Material and Software

- Open Source Structure-from-Motion tutorial at CVPR 2015
  - http://www.kitware.com/cvpr2015-tutorial.html
- Advanced notes on bundle adjustment
- Tutorials on several popular open source SfM packages